Environment Capability Assessment: Can This Sandbox Run Accelerated Inference for `spatialformer`?

Date: 2026-08-02 · **Method:** every answer determined empirically inside the `execute_code` tool (Python 3.12.13, Linux `x86_64`).

Summary

Short answer: No — this environment cannot run *accelerated* (GPU/TPU) inference, because it has no hardware accelerator of any kind. It is a CPU-only Linux sandbox. A freshly installed PyTorch build (`torch 2.13.0+cpu`) reports `torch.cuda.is_available() == False`, `torch.cuda.device_count() == 0`, and no Metal/MPS backend. There are no NVIDIA device nodes (`/dev/nvidia*`), no AMD ROCm compute node (`/dev/kfd`), no `/proc/driver/nvidia/version`, and `CUDA_VISIBLE_DEVICES` is unset. All numerical work would therefore fall to the 4 available CPU cores.

However, the setup ingredients for *CPU-only* inference all exist. The sandbox has 4 virtual CPU cores (Intel Xeon Platinum 8375C @ 2.90 GHz), ~16 GB of system RAM (~10.5 GiB available), and ~129 GB of free working disk (out of 415 GB). It has working outbound internet: the Python Package Index (PyPI), Figshare (both the API host and the file host), the NCBI Gene Expression Omnibus (GEO), and Hugging Face all responded with HTTP success codes. The target package installs from PyPI — `spatialformer 0.1.8`, a pure-Python wheel requiring Python ≥3.10 — and the environment runs Python 3.12.13, which satisfies that constraint. Once PyTorch is present, the package imports and exposes its analysis functions (e.g., `embed_data`, `create_dataloader_eval`).

The binding constraint is not capability but orchestration. The `execute_code` sandbox is stateless between calls and enforces a per-call wall-clock limit of roughly 60 seconds. Installing `spatialformer` *with* its full dependency tree resolves 116 packages (including heavy CUDA `nvidia_*` wheels pulled in by the default PyTorch metadata) and does not finish within the limit. A full workflow — install PyTorch + all dependencies, download a multi-gigabyte weights file from Figshare, load the model, and run it — therefore cannot be chained inside a single

call as the tool is currently exposed. It would need to be broken into stages against a persistent working directory, or run in an environment without the short per-call timeout. In principle CPU inference is feasible; in practice it requires a staged execution model that this specific sandbox interface does not natively provide within one call.

Answers to the Seven Questions

#	Question	Observed answer	How determined
1	Hardware accelerator?	**None.** No GPU/TPU/accelerator present. CPU-only.	Checked for `/dev/nvidia*`, `/dev/kfd`, `/proc/driver/nvidia/version` (all absent); `CUDA_VISIBLE_DEVICES` unset; `torch.cuda.is_available()=False`, `device_count()=0`, `backends.mps.is_available()=False`.
2	Cores / RAM / free disk	**4 CPU cores; ~16 GB RAM (~15.3 GiB total, ~10.5 GiB available); ~129 GB free disk** (of ~415 GB).	`os.cpu_count()=4` (Intel Xeon Platinum 8375C @2.90 GHz); `/proc/meminfo` `MemTotal=16,080,212 kB`; `os.statvfs('/')` → 129.4 GB free of 414.9 GB.
3	PyTorch present? Accelerator usable?	**Not pre-installed.** After `pip install`, `torch 2.13.0+cpu` installs cleanly (~40 s) but reports **no usable accelerator** (`cuda.is_available()=False`, CPU matmul works).	Filesystem scan of `site-packages` (no `torch`); installed CPU build from PyTorch CPU index; queried CUDA/MPS availability in a child process.
4	Internet reachable? (PyPI / Figshare / GEO / Hugging Face)	**All reachable.** PyPI `200`, Figshare API `200` / site `202`, GEO (NCBI) `200`, Hugging Face `200`. PyTorch CPU index also reachable.	HTTP GET via `requests` per host.
5	Can you install `spatialformer`? Which version?	**Yes** — **`spatialformer 0.1.8`** (pure-Python wheel, 10.9 MB, `requires_python>=3.10`). Installs on Python 3.12.13.	`pip install --no-deps spatialformer` → "Successfully installed spatialformer-0.1.8". Full install *with* deps exceeded the ~60 s per-call limit (RC=124).
6	Analysis functions exposed	`tools` (`tl`), `create_dataloader_eval`, `create_single_data_loaders`; `tools.get_embeddings` exposes `embed_data`, `valid_mean_embedding`, `reveal_gene_pairs`, `process_bidirectional_predictions`, `prepare_extended_checkpoint`, `manual_train_fm`; plus model class	Inspected `__init__` and `tools/get_embeddings.py` after install; a real `import spatialformer` requires `torch`, then `pytorch_lightning`.

#	Question	Observed answer	How determined
		`Spaformer`, `Processor`, `GeneInteractionProcessor`.	
7	Approx. wall-clock time	**~19–20 minutes** for the whole session.	`time.time()` delta from first probe; per-call ceiling is ~60 s (torch install ~40 s; guarded full install hit the 55 s guard).

Key Findings

Finding 1 — No hardware accelerator; a modest CPU-only Linux sandbox

The environment is a **CPU-only Linux sandbox with no accelerator of any kind**. This was established negatively and positively. Negatively: none of the usual accelerator device nodes or driver interfaces exist — there is no `/dev/nvidia*` (NVIDIA GPU), no `/dev/kfd` (AMD ROCm compute), and no `/proc/driver/nvidia/version`. The `CUDA_VISIBLE_DEVICES` environment variable is unset. Positively: a freshly installed PyTorch build (`torch 2.13.0+cpu`, the CPU-only wheel) directly reports `torch.cuda.is_available() == False`, `torch.cuda.device_count() == 0`, and `torch.backends.mps.is_available() == False`. In other words, the deep-learning framework itself confirms there is nothing to accelerate on, while a CPU matmul executed correctly.

The compute resources are modest but real: `os.cpu_count()` returns **4** (an Intel Xeon Platinum 8375C @ 2.90 GHz), `/proc/meminfo` shows `MemTotal = 16,080,212 kB` (~15.3 GiB, with ~10.5 GiB available at inspection time), and `os.statvfs('/')` reports **129.4 GB free** of a 414.9 GB filesystem. The disk headroom is comfortable for a multi-gigabyte weights file; the RAM (~10.5 GiB available) is the tighter budget for loading a large transformer model on CPU, though a several-hundred-MB-to-few-GB checkpoint would typically fit.

Implication: Any inference here is strictly CPU inference. For a transformer-class spatial-transcriptomics model, CPU inference is possible but slow — often one to two orders of magnitude slower than GPU — and is sensitive to the RAM ceiling for large batches.

Finding 2 — Public internet is reachable; `spatialformer 0.1.8` installs and exposes its functions

All four externally hosted resources named in the brief are reachable. HTTP GET requests via the `requests` library returned success codes for: the PyPI simple index (`pypi.org`, `200`) and the `spatialformer` project JSON; the Figshare API host (`api.figshare.com`, `200`) and the Figshare file/web host (`figshare.com`, `202`); the NCBI GEO landing (`ncbi.nlm.nih.gov`, `200`); Hugging Face (`huggingface.co`, `200`); and the PyTorch CPU wheel index (`download.pytorch.org`). This confirms the network paths required to (a) install the package, (b) download pretrained weights from Figshare, and (c) fetch a Xenium-derived dataset from GEO are all open.

The package metadata on PyPI describes `spatialformer 0.1.8` as a **pure-Python wheel (10.9 MB)** with `requires_python >= 3.10`. The sandbox runs **Python 3.12.13**, satisfying that requirement. A dependency-free install (`pip install --no-deps spatialformer`) succeeded ("Successfully installed spatialformer-0.1.8"). Inspecting the installed package, the top-level `__init__` exposes `tools` (aliased `tl`), `create_dataloader_eval`, and `create_single_data_loaders`; the `tools.get_embeddings` module exposes the analysis functions `embed_data`, `valid_mean_embedding`, `reveal_gene_pairs`, `process_bidirectional_predictions`, `prepare_extended_checkpoint`, and `manual_train_fm`, alongside the model class `Spaformer`, a `Processor`, and a `GeneInteractionProcessor`. A genuine `import spatialformer` (in a child process) initially failed with `ModuleNotFoundError: No module named 'torch'` at `tools/get_embeddings.py:31`, confirming that the package is a thin analysis layer over PyTorch and cannot be imported until PyTorch is present. Separately, `torch 2.13.0+cpu` installed cleanly from the PyTorch CPU index in about 40 seconds.

Implication: Every static prerequisite for CPU inference is satisfiable — the package installs, its API is discoverable, and the model's data/weights hosts are reachable.

Finding 3 — The full dependency chain does not fit inside one ~60 s stateless call

The practical blocker is orchestration, not capability. Installing `spatialformer` *with* its full dependency tree (rather than `--no-deps`) triggers a large resolution: pip (25.0.1) resolved **116 dependencies** and was still downloading large wheels — including heavy CUDA `nvidia_*` wheels (e.g., `nvidia_nvjitlink` at 40.7 MB) pulled in by the default PyTorch metadata — when a `timeout 55` guard killed the process (return code 124). "Successfully installed" was never reached within the window. Even with a CPU `torch` pre-installed, a real `import`

`spatialformer` advanced past the `torch` import and then failed at `spatialformer/model/Spaformer_pair.py:5: import pytorch_lightning` (`ModuleNotFoundError`), revealing a **layered heavy-dependency chain**: `torch → pytorch_lightning → transformers/datasets/...`.

Because the `execute_code` sandbox is **stateless between calls** (files in `/tmp` and background processes do not persist across calls) and enforces a **~60 s per-call ceiling**, the sequence "install torch + pytorch_lightning + transformers + datasets, then download a multi-GB Figshare checkpoint, then load and run the model" cannot be completed inside a single call. Each stage individually is fine; chaining them is what exceeds the limit.

Implication: To actually run inference, the work must be staged — e.g., install dependencies into a *persistent* target directory in one or more calls, download weights in another, then import and run — or executed in an environment without the short per-call timeout. The default PyTorch index also needlessly pulls CUDA wheels; forcing the CPU index (`--index-url https://download.pytorch.org/whl/cpu`) would shrink the download substantially and avoid useless GPU packages on this CPU-only host.

Interpretation: A Capability Map

The environment can be summarized as a stack of "can" layers sitting on one "cannot" foundation:

```
                  CAN this environment ...?

Run GPU/TPU-accelerated inference   ──►  NO  (no accelerator hardware at all)
──────────────────────────────────────────────────────────────────────────────
Provide CPU compute (4 cores/16 GB) ──►  YES
Provide ~129 GB free disk           ──►  YES (fits multi-GB weights)
Reach PyPI                          ──►  YES (install packages)
Reach Figshare (API + files)        ──►  YES (weights host)
Reach GEO                           ──►  YES (Xenium dataset host)
Reach Hugging Face                  ──►  YES
Install spatialformer 0.1.8         ──►  YES (pure-Python, needs Py>=3.10; have 3.12)
Install CPU PyTorch                 ──►  YES (torch 2.13.0+cpu, ~40 s)
Import package & see its API        ──►  YES (once torch present)
──────────────────────────────────────────────────────────────────────────────
Chain full install + multi-GB DL    ──►  NOT in one ~60 s stateless call
+ model run in a SINGLE call             (needs staged/persistent execution)
```

The distinction that matters for the user's underlying goal is **"accelerated" vs "feasible."** Accelerated inference is categorically off the table — there is no GPU. CPU-only inference is *set-up-feasible*: every ingredient (package, framework, network reachability, disk) is present and verified. What stands between "feasible" and "done" is the per-call time budget, which forces a staged execution pattern rather than a one-shot script.

Dependency chain observed

```
spatialformer 0.1.8  (pure-Python, 10.9 MB wheel)
        │
        ├── import torch                ──►  required first (else ModuleNotFoundError)
        │
        └── import pytorch_lightning    ──►  required next (Spaformer_pair.py:5)
                  │
                  └── transformers / datasets / ...  (part of 116-package resolution)
```

Evidence Base

This is an empirical self-assessment of execution capabilities rather than a literature review, so the "evidence base" is the set of observations recorded during the two investigation iterations rather than published papers. No PubMed citations are applicable to a hardware/software capability audit. The primary evidence is:

- **Kernel/device inspection** (absence of `/dev/nvidia*`, `/dev/kfd`, `/proc/driver/nvidia/version`; unset `CUDA_VISIBLE_DEVICES`) → establishes no accelerator.
- **PyTorch self-report** (`torch 2.13.0+cpu`, `cuda.is_available()=False`, `device_count()=0`, `mps=False`, CPU matmul OK) → confirms no usable accelerator at the framework level.
- **System introspection** (`os.cpu_count()`, `/proc/cpuinfo`, `/proc/meminfo`, `os.statvfs`) → cores, RAM, disk.
- **HTTP reachability probes** (200/202 responses from PyPI, Figshare API and file host, GEO, Hugging Face, PyTorch CPU index) → network capability.
- **pip install transcripts** (`--no-deps` success; full-dep timeout RC=124; 116-package resolution; CUDA `nvidia_*` wheels) → install feasibility and its limits.
- **Package introspection** (`__init__` and `tools/get_embeddings.py` symbols; import failures at `torch` then `pytorch_lightning`) → API surface and dependency ordering.

Limitations and Knowledge Gaps

1. **The full install was never completed**, only characterized up to a timeout. The complete set of transitive dependencies (beyond the 116 resolved) and their total on-disk footprint were not measured to completion. We know the chain includes `torch`, `pytorch_lightning`, `transformers`, and `datasets`, but not the final installed size.
2. **No model was loaded or run** (per the brief's explicit constraint). Therefore actual CPU inference latency, memory peak during a forward pass, and whether a several-GB checkpoint fits within ~10.5 GiB available RAM were not empirically measured — they are inferred as "plausible but untested."
3. **Sandbox import allow-list.** The interactive `execute_code` layer blocks some direct imports (e.g., `torch`, `subprocess`, `sys`); accelerator and torch checks were therefore run via child processes and filesystem inspection, which faithfully reflect the underlying machine but add a layer of indirection.
4. **Persistence across calls was not exhaustively mapped.** We established that files in `/tmp` and background processes do not persist between calls; whether a target install directory on a shared filesystem reliably persists across calls (which would enable staged installs) was inferred but not stress-tested end-to-end.
5. **Figshare/GEO reachability was verified at the landing/API level**, not by completing an actual multi-GB download (per the brief). Reachability was confirmed via HTTPS GET, but sustained large-file throughput and any rate-limiting were not measured.
6. **PyTorch version drift.** The installed `torch 2.13.0+cpu` was the current wheel at session time; `spatialformer` pins were not audited for compatibility, so a version conflict during a full dependency resolution remains possible.

Proposed Follow-up Actions

If the goal is to actually stand up CPU inference in a follow-up session, the following concrete steps are recommended:

1. **Stage the install against a persistent directory.** Run pip in separate calls that write to a fixed `--target`/venv path on a shared filesystem, so partial progress survives between calls. Split into: (a) `torch` + `pytorch_lightning`, (b) `transformers` + `datasets` + remaining deps, (c) `spatialformer` itself.
2. **Force the CPU wheel index for PyTorch** with `pip install torch --index-url https://download.pytorch.org/whl/cpu` to avoid pulling the large CUDA `nvidia_*` wheels that are useless on this accelerator-free host. This both shrinks download volume and reduces install time.
3. **Pre-flight the weights download separately.** Issue a Figshare API request to resolve the exact checkpoint URL and size, then download in a dedicated call (or a resumable, chunked download) to the ~129 GB of free disk — never inside the same call as the install.
4. **Measure the RAM peak on a tiny synthetic AnnData** before attempting a real Xenium slide, to confirm the model + checkpoint fit within ~10.5 GiB and to size batch limits for CPU.
5. **Benchmark CPU forward-pass latency** on a small number of cells to produce a realistic throughput estimate (cells/second on 4 cores), which will determine whether processing a full Xenium slide is practical or needs subsetting.
6. **If accelerated inference is truly required**, provision a different environment with a GPU — this sandbox cannot be made to accelerate regardless of software configuration, because the hardware is absent.

Explicit Verdict

Can this environment run *accelerated* inference for a PyTorch model that first needs a multi-gigabyte weights download from Figshare? No. There is **no accelerator present** (CPU-only; `torch.cuda.is_available()=False`, 0 devices). Free resources are **~129 GB of disk** (ample for multi-GB weights) and **~10.5 GiB of available RAM on 4 CPU cores** — but **zero accelerator memory**, because there is no accelerator. **Figshare (API + file host) and GEO are both reachable** (HTTP 200/202), as are PyPI and Hugging Face.

Is processor-only inference feasible? Feasible in principle, with a staged setup. The three enabling conditions all hold: **PyTorch installs** (`torch 2.13.0+cpu`, CPU build), **the package imports and exposes its functions** once torch (and `pytorch_lightning`) are present (`spatialformer 0.1.8`, `embed_data`, `create_dataloader_eval`, etc.), and **the required downloads are reachable** (PyPI, Figshare, GEO). The one caveat is operational, not capability-based: the full dependency install plus a multi-GB weights download plus a model run **cannot be chained inside a single ~60 s stateless `execute_code` call** and must be broken into stages against a persistent working directory.
